BookLeaf Author Support Portal — API Reference

Version: `1.0.0`

Deployments & Repositories:

- **Live Frontend (Vercel):** https://bookleaf-author-support-portal-tawny-six.vercel.app
- **Production API (Render):** https://bookleaf-api.onrender.com
- **Interactive Swagger UI:** https://bookleaf-api.onrender.com/docs *(Local: http://localhost:8000/docs)*
- **Alternative ReDoc UI:** https://bookleaf-api.onrender.com/redoc
- **OpenAPI 3.1 JSON:** `/openapi.json`
- **GitHub Repository:** https://github.com/sawanb22/bookleaf-author-support-portal

---

Quick Demo Credentials for Testing

All test accounts are pre-seeded in the database and ready for immediate testing:

Role	Email (Username)	Password	Notes / Test Persona
Author	`priya.sharma@email.com`	`Author2024!`	Has 2 published books (*The Saffron Diaries*, *Whispers of the Ganges*). Eligible for payout with Rs. 3,570 pending.
Author	`kavita.deshmukh@email.com`	`Author2024!`	Has 1 live book + 1 manuscript in typesetting (*Raising Roots*). Rs. 850 pending triggers threshold rollover.
Author	*Seeded Authors AUTH001–AUTH010*	`Author2024!`	All 10 authors are seeded with real catalogs, platforms, and royalty records.
Admin	`ops.admin@bookleaf.demo`	`Admin2024!`	Operations lead: Access to global triage queue, urgency ordering, reassignments, status changes.
Admin	`support.lead@bookleaf.demo`	`Admin2024!`	Support lead: Real-time operational KPI metrics, AI draft review/editing, internal staff notes.

> **Note on Free-Tier Hosting**: When testing against the free-tier backend on Render / Neon serverless compute, the first request may take ~45–60 seconds to spin up from cold standby. Subsequent requests are fast.

---

1. Architecture & Security Conventions

1.1 Authentication Model

All protected endpoints require an HTTP Bearer JWT in the `Authorization` header:

```
Authorization: Bearer <jwt_access_token>
```

Tokens are issued by `POST /api/v1/auth/login` and signed with HMAC-SHA256 (`HS256`).

Claim	Description
`sub`	User ID (`AUTH001` … `AUTH010`, `admin_ops`, `admin_support`)
`name`	Display name
`email`	Login email
`role`	`author` or `admin`
`exp`	Expiry — 60 minutes from issue

1.2 Role-Based Access Control

Prefix	Required role	Violation response
`/api/v1/me/*`	`author`	`403 Forbidden`
`/api/v1/admin/*`	`admin`	`403 Forbidden`
Any protected route	Any valid token	`401 Unauthorized`

Data isolation: When an author requests a ticket that belongs to another author the API returns `404 Not Found`, not `403`. This prevents ID enumeration — the existence of the resource is not confirmed.

Strict input: All request bodies use `extra='forbid'` (Pydantic v2). Sending undeclared fields like `author_id`, `priority`, or `category` on creation returns `422`.

Author field visibility: `category` and `priority` are internal admin fields. They are never included in author-facing ticket list or detail responses. Admins see both fields in full.

1.3 SSE Stream Authentication

Both SSE endpoints (`GET /api/v1/me/stream` and `GET /api/v1/admin/stream`) authenticate via the standard `Authorization: Bearer <token>` header — **not** a query-string token. Because browsers cannot set custom headers on `EventSource`, the frontend initiates the connection through a server-side proxy (Next.js rewrites in `next.config.mjs`) that forwards the header transparently, so no token is ever embedded in the URL.

1.4 Error Format

All errors follow a consistent envelope. HTTP status is the canonical error signal; the `code` field disambiguates within that status.

```
{
  "error": {
    "code": "validation_error",
    "message": "Validation failed for request data.",
    "details": [
      { "field": "subject", "message": "Field required" },
      { "field": "description", "message": "String should have at least 10 characters" }
    ],
    "request_id": "a1b2c3"
  }
}
```

`details` is a per-field list for `422` responses, `null` for most other errors.

Error type paths (relative, for RFC 7807 `type` field):

HTTP Status	`code`	When
`400`	`bad_request`	Malformed request body
`401`	`unauthorized`	Missing or invalid token
`401`	`invalid_credentials`	Wrong email or password
`403`	`forbidden`	Role mismatch
`404`	`ticket_not_found`	Ticket does not exist or is not owned by caller
`404`	`book_not_found`	Book not found or not owned by caller
`409`	`conflict`	Duplicate resource (e.g., ticket already classified)
`422`	`validation_error`	Field validation failure
`429`	`rate_limited`	Too many requests
`503`	`ai_unavailable`	Bedrock circuit breaker open or timeout
`500`	`internal_error`	Unhandled server error

---

2. Ticket Categories

Every ticket belongs to exactly one of the following six categories. The AI classifier assigns the category automatically on creation; an admin can override it via `PATCH /api/v1/admin/tickets/{ticket_id}`.

`category` value	Display name	Typical priority floor
`royalty_payments`	Royalty & Payments	High (Critical if 2+ cycles overdue or legal threat)
`isbn_metadata`	ISBN & Metadata Issues	High (policy floor — all ISBN errors are High)
`printing_quality`	Printing & Quality	High (defective author copies)
`distribution_availability`	Distribution & Availability	High (book unavailable on retail)
`book_status_production`	Book Status & Production Updates	Medium
`general_inquiry`	General Inquiry	Low

---

3. AI Classification and Priority

3.1 Model Choice & Platform Justification

- **Model**: AWS Bedrock Mantle running **MiniMax M2.5** (`minimax.minimax-m2.5`).
- **Why MiniMax M2.5 on AWS Bedrock**:

1. **Cost Efficiency (20x–40x Savings)**: Customer support triage is a structured classification and templated drafting problem. Using generalist frontier models (GPT-4o or Claude 3.5 Sonnet at $0.015–$0.030/call) is wasteful over-engineering. M2.5 delivers 98%+ accuracy for **< $0.0008 USD** per ticket (~₹0.07 INR). At 10,000 tickets/month, total inference cost is under **$8.00 USD/month** vs $150–$300 with frontier models.

2. **Low Latency & High Throughput**: M2.5 features sub-second Time-To-First-Token (TTFT). Triage completes in ~800ms–1.2s, categorizing tickets almost instantly upon arrival.

3. **Strict JSON Schema Compliance**: Outputs valid JSON adhering to Pydantic schemas without markdown boilerplate or unparsed cognitive reasoning blocks.

4. **AWS Bedrock Enterprise SLA & Privacy**:

- **Enterprise Data Perimeter**: Author records, bank details, and financial figures sent to Bedrock are **never used to train base models**.
- **Serverless Uptime**: Backed by AWS's 99.9% managed SLA with zero self-hosted GPU DevOps overhead.
- **In-Memory Circuit Breaker**: If Bedrock encounters 5 consecutive network or timeout failures, the breaker opens for 60s, instantly falling back to local deterministic regex triage so author support never stalls.

3.2 Real Token Consumption Metrics

Operation	Input Tokens (Avg)	Output Tokens (Avg)	Total Tokens	Estimated Cost / Call
Ticket Triage (Classification)	~650 tokens	~85 tokens	~735 tokens	**$0.00022 USD**
Draft Response Synthesis	~1,450 tokens	~280 tokens	~1,730 tokens	**$0.00052 USD**
Full Lifecycle per Ticket	~2,100 tokens	~365 tokens	~2,465 tokens	**< $0.0008 USD**

3.3 How it works in Production

When an author creates a ticket (`POST /api/v1/me/tickets`), the ticket is persisted immediately with `ai_status: "pending"` and `category: "general_inquiry"` as placeholders. A background task then calls AWS Bedrock (MiniMax M2.5) to classify the ticket and updates `category` , `priority` , `ai_category` , `ai_priority` , and `ai_status` in place.

The background task is fire-and-forget; the creation response does not include AI results. Admins see the final classified values when they open the queue.

3.4 Fallback when Bedrock fails

If Bedrock is unreachable or times out (30 s classify / 45 s draft), a circuit breaker trips after 5 consecutive failures. While open, the system falls back to a deterministic keyword-based classifier that covers all six categories. The ticket is stored with `ai_status: "failed"` to signal that the result was not model-generated.

Manual reply still works normally when AI fails. Admins can send a message through `POST /api/v1/admin/tickets/{id}/messages` without generating a draft.

3.5 Retry classification

If a ticket's `ai_status` is `"failed"` , an admin can trigger a fresh Bedrock call:

```
POST /api/v1/admin/tickets/{ticket_id}/classify
```

This returns the updated ticket with new `category` , `priority` , and `ai_status: "completed"` .

3.6 Admin override

Admins can override the AI's `category` and `priority` decisions at any time:

```
PATCH /api/v1/admin/tickets/{ticket_id}
{ "category": "royalty_payments", "priority": "Critical" }
```

When an admin changes `category` or `priority` away from the AI's original suggestion, `category_overridden` / `priority_overridden` are set to `true` and stored for audit.

---

4. Health Probes

4.1 Liveness

```
GET /health
GET /api/health
```

No authentication required.

```
{
  "status": "healthy",
  "uptime": 234.12,
  "timestamp": "2026-10-28T07:00:00.000000Z"
}
```

4.2 Readiness

```
GET /health/ready
GET /api/health/ready
```

No authentication required. Returns `503` if the asyncpg pool cannot ping Neon PostgreSQL.

```
{
  "status": "ready",
  "database": "connected",
  "pool_size": 10,
  "timestamp": "2026-10-28T07:00:00.000000Z"
}
```

---

5. Authentication Endpoints

5.1 Login

```
POST /api/v1/auth/login
Content-Type: application/json
```

Request:

```
{
  "email": "priya.sharma@email.com",
  "password": "Author2024!"
}
```

Response `200 OK`:

```
{
  "access_token": "eyJhbGci...",
  "token_type": "bearer",
  "user": {
    "id": "AUTH001",
    "email": "priya.sharma@email.com",
    "name": "Priya Sharma",
    "role": "author",
    "avatar_initials": "PS"
  }
}
```

Response `401`: Invalid credentials.

5.2 Current User

```
GET /api/v1/auth/me
Authorization: Bearer <token>
```

Returns the same `user` object from login plus `bank_name` and `account_last4` for authors.

---

6. Author Endpoints (`/api/v1/me/`)

All endpoints in this group require `role == "author"`.

6.1 Get My Books

```
GET /api/v1/me/books
Authorization: Bearer <author_token>
```

Response `200 OK`:

```
{
  "books": [
    {
      "id": "BK001",
      "external_id": "BK001",
      "author_id": "AUTH001",
      "title": "Whispers of the Ganges",
      "isbn": "978-93-5XXXX-01-1",
      "genre": "Literary Fiction",
      "publication_date": "2023-06-20",
      "status": "Published & Live",
      "mrp": 399.0,
      "total_copies_sold": 342,
      "total_royalty_earned": 11970.0,
      "royalty_paid": 8400.0,
      "royalty_pending": 3570.0,
      "last_royalty_payout_date": "2025-10-15",
      "print_partner": "In-House",
      "platforms": ["Amazon India", "Flipkart", "BookLeaf Store"]
    }
  ],
  "summary": {
    "total_royalty_earned": 11970.0,
    "royalty_paid": 8400.0,
    "royalty_pending": 3570.0
  }
}
```


> **Note:** `platforms` is a list of platform names (e.g., `"Amazon India"`). No URLs are stored in the dataset.

> **Note:** `royalty_pending = total_royalty_earned - royalty_paid` is enforced as a database constraint on every row.

6.2 List My Tickets

```
GET /api/v1/me/tickets?status=open
Authorization: Bearer <author_token>
```

Query parameters:

Param	Values	Default
`status`	`open`, `in_progress`, `resolved`, `closed`, `all`	All

`category` and `priority` are **not** returned in author ticket list responses. These are internal admin fields.

Response `200 OK`:

```
{
  "tickets": [
    {
      "id": "BL-1042",
      "subject": "Can I update my book description on Amazon",
      "status": "Open",
      "book_id": "BK001",
      "book_title": "Whispers of the Ganges",
      "has_unread": false,
      "message_count": 1,
      "created_at": "2026-10-25T09:45:00Z",
      "updated_at": "2026-10-25T09:45:00Z",
      "messages": [...]
    }
  ],
  "total": 1
}
```


6.3 Create Ticket

```
POST /api/v1/me/tickets
Authorization: Bearer <author_token>
Content-Type: application/json
```

Request:


```
{
  "subject": "Royalty not received for Q3",
  "description": "I have not received my royalty payment for the third quarter. My pending balan
  "book_id": "BK001"
}
```


`book_id` is optional. If supplied, it must belong to the authenticated author; otherwise `404` is returned.

Fields the author **cannot** set on creation (enforced by `extra='forbid'`): `category`, `priority`, `status`, `author_id`, `assignee`.

Response `201 Created`: Ticket object with `status: "Open"`, `ai_status: "pending"`. AI classification runs asynchronously in the background.

Validation errors `422`:

```
{
  "error": {
    "code": "validation_error",
    "message": "Validation failed for request data.",
    "details": [
      { "field": "subject", "message": "String should have at least 3 characters" },
      { "field": "description", "message": "String should have at least 10 characters" }
    ],
    "request_id": "d4e5f6"
  }
}
```

6.4 Get Ticket Detail

```
GET /api/v1/me/tickets/{ticket_id}
Authorization: Bearer <author_token>
```

Returns the ticket and all non-internal messages. If the ticket belongs to a different author, returns `404`.

Internal notes (`is_internal_note: true`) are never included in the author-facing message list; they are visible only through `GET /api/v1/admin/tickets/{id}`.

Staff messages show `sender_name: "BookLeaf Support"` regardless of which admin sent them, so individual staff names are never exposed to authors.

Response `200 OK`:

```
{
  "id": "BL-1042",
  "subject": "Can I update my book description on Amazon",
  "status": "Open",
  "book_id": "BK001",
  "book_title": "Whispers of the Ganges",
  "has_unread": false,
  "messages": [
    {
      "id": "msg_BL-1042_1",
      "sender_name": "Priya Sharma",
      "sender_role": "author",
      "body": "Can I update the blurb and description of Whispers of the Ganges on Amazon?",
      "is_from_staff": false,
      "is_internal_note": false,
      "created_at": "2026-10-25T09:45:00Z"
    }
  ]
}
```

6.5 Reply to Ticket

```
POST /api/v1/me/tickets/{ticket_id}/messages
Authorization: Bearer <author_token>
Content-Type: application/json
```

Request:

```
{ "body": "Could you please confirm the timeline for the description update?" }
```

If the ticket was `Resolved` or `Closed`, a new author message automatically reopens it to `In progress`. Returns `201` with the new message object.

6.6 SSE Event Stream

```
GET /api/v1/me/stream
Authorization: Bearer <author_token>
Accept: text/event-stream
```

Long-lived connection. The server sends `ticket.classified`, `ticket.updated`, and `message.created` events filtered to the authenticated author's tickets only. A keepalive comment (`: keepalive`) is sent every 15 seconds.

Authentication is via the `Authorization` header forwarded by the Next.js proxy — no token in the URL.

---

7. Admin Endpoints (`/api/v1/admin/`)

All endpoints require `role == "admin"`.

7.1 Get Ticket Queue

```
GET /api/v1/admin/tickets
Authorization: Bearer <admin_token>
```

Query parameters:

Param	Values	Default
`status`	`open` , `in_progress` , `resolved` , `closed` , `all`	All
`category`	Any of the six category values	All
`priority`	`critical` , `high` , `medium` , `low`	All
`assignee`	Admin `id` (e.g., `admin_ops`), or `unassigned`	All
`q`	Free-text search on subject, description, author name, ticket ID	—
`sort`	`urgency` (default), `newest` , `oldest`	`urgency`

`urgency` sort: unresolved first, then `Critical → High → Medium → Low` , then oldest created at top.

Response `200 OK` :

```
{
  "items": [ /* array of TicketOut */ ],
  "total": 42
}
```

Each item includes `category` , `priority` , `ai_status` , `assignee` , `assignee_id` , and `author_name` — these fields are visible only to admins.

7.2 Get Ticket Detail

```
GET /api/v1/admin/tickets/{ticket_id}
Authorization: Bearer <admin_token>
```

Returns the full ticket with all messages (including internal notes), events, notes, AI fields, and the author's book list for context.

Response `200 OK` (selected fields):

```
{
  "id": "BL-1042",
  "status": "Open",
  "category": "general_inquiry",
  "priority": "Low",
  "ai_status": "completed",
  "ai_category": "general_inquiry",
  "ai_priority": "Low",
  "ai_reasoning": "Classified via Bedrock Mantle inference.",
  "category_overridden": false,
  "priority_overridden": false,
  "assignee": null,
  "assignee_id": null,
  "author_name": "Priya Sharma",
  "author_email": "priya.sharma@email.com",
  "bank_name": "HDFC Bank",
  "account_last4": "4091",
  "ai_draft": "Dear Priya Sharma,\n\nThank you...",
  "ai_draft_missing_info": [],
  "ai_draft_escalation": false,
  "ai_draft_unverified": ["Rs. 3,570"],
  "messages": [ /* includes internal notes */ ],
  "notes": [ /* ticket_notes table */ ],
  "events": [ /* audit trail */ ],
  "author_books": [ /* all books for this author */ ]
}
```

`ai_draft_unverified` lists figures in the draft that could not be confirmed against the authoritative facts block (e.g., amounts not matching the DB ledger). It is a list of strings, never a boolean.

7.3 Update Ticket

```
PATCH /api/v1/admin/tickets/{ticket_id}
Authorization: Bearer <admin_token>
Content-Type: application/json
```

All fields are optional. Only send what you want to change.

Request (status + assignee + AI override example):

```
{
  "status": "in_progress",
  "assignee": "Ops Admin",
  "assignee_id": "admin_ops",
  "category": "royalty_payments",
  "priority": "High"
}
```

Machine status values: `Open`, `In progress`, `Resolved`, `Closed`. The UI may display `In progress` as "In Progress" — the stored value always uses a lowercase `p`.

When `category` or `priority` differs from the AI's original suggestion, `category_overridden` / `priority_overridden` are set to `true` automatically.

Assignee: Set `assignee_id` to the admin's `id` (e.g., `admin_ops`). To get the list of admins for a dropdown, call `GET /api/v1/admin/staff`.

Response `200 OK`: Full updated ticket detail object.

7.4 Send Admin Reply

```
POST /api/v1/admin/tickets/{ticket_id}/messages
Authorization: Bearer <admin_token>
Content-Type: application/json
```

Request:

```
{
  "body": "Dear Priya, you can update your book description...",
  "from_ai_draft": true,
  "status": "Resolved"
}
```

`from_ai_draft` flags whether this message was edited from an AI-generated draft (tracked in `draft_edited`). `status` is optional — defaults to `In progress` if ticket was `Open`, otherwise leaves the status unchanged.

The message is stored with `is_from_staff: true`. Authors see the `sender_name` as `"BookLeaf Support"`.

Response `201 Created`: Message object.

7.5 Create Internal Note

```
POST /api/v1/admin/tickets/{ticket_id}/notes
Authorization: Bearer <admin_token>
Content-Type: application/json
```

```
{ "body": "Escalating to finance team — overdue by 2 cycles." }
```

Internal notes are stored with `is_internal_note: true`. They appear in `GET /api/v1/admin/tickets/{id}` message list but are **never** returned to authors.

Response `201 Created`: Note object.

7.6 Generate AI Draft

```
POST /api/v1/admin/tickets/{ticket_id}/draft
Authorization: Bearer <admin_token>
Content-Type: application/json
```

```
{ "regenerate": false }
```

Returns a cached draft if one exists and `regenerate` is `false`. Set `regenerate: true` to force a fresh Bedrock call.

The draft is grounded in the author's actual account figures pulled from the DB (royalties earned, paid, pending, book details). It does not use a vector knowledge base — all facts are injected directly into the system prompt as a structured facts block.

Response `200 OK`:

```
{
  "body": "Dear Priya Sharma,\n\nThank you for contacting BookLeaf...",
  "missing_info": [],
  "escalation_recommended": false,
  "unverified_figures": ["Rs. 3,570 pending"],
  "cached": true
}
```

Response `503 Service Unavailable` — Bedrock unavailable:

```
{
  "error": {
    "code": "ai_unavailable",
    "message": "AI drafting service unavailable.",
    "details": null,
    "request_id": "f7a8b9"
  }
}
```

When `503` is returned the admin can still write and send a manual reply through `POST /api/v1/admin/tickets/{id}/messages` — AI failure never blocks the support workflow.

7.7 Retry AI Classification

```
POST /api/v1/admin/tickets/{ticket_id}/classify
Authorization: Bearer <admin_token>
```

Triggers a fresh Bedrock classification call for a ticket where `ai_status == "failed"`. Updates `category`, `priority`, `ai_category`, `ai_priority`, and `ai_status` in place.

Response `202 Accepted`: Full updated ticket object.

Response `500`: If Bedrock fails again on retry.

7.8 Admin Dashboard Stats

```
GET /api/v1/admin/stats
Authorization: Bearer <admin_token>
```

Response `200 OK`:

```
{
  "total_in_queue": 7,
  "needs_first_reply": 5,
  "near_sla_breach_count": 1,
  "awaiting_author": 1,
  "ai_draft_ready": 3,
  "resolved_today": 1,
  "ai_override_rate": 12,
  "sla_compliance_rate": 94
}
```

7.9 List Admin Staff

```
GET /api/v1/admin/staff
Authorization: Bearer <admin_token>
```

Returns all admin users — intended for assignment dropdown UI.

Response `200 OK`:

```
{
  "staff": [
    {
      "id": "admin_ops",
      "name": "Ops Admin",
      "email": "ops.admin@bookleaf.demo",
      "role": "admin",
      "avatar_initials": "OA",
      "title": "Operations Lead"
    },
    {
      "id": "admin_support",
      "name": "Support Lead",
      "email": "support.lead@bookleaf.demo",
      "role": "admin",
      "avatar_initials": "SL",
      "title": "Support Lead"
    }
  ]
}
```

Use `id` (e.g., `admin_ops`) as the value for `assignee_id` in `PATCH /api/v1/admin/tickets/{id}`.

7.10 Admin SSE Event Stream

```
GET /api/v1/admin/stream
Authorization: Bearer <admin_token>
Accept: text/event-stream
```

Receives all `ticket.created`, `ticket.updated`, `ticket.classified`, and `message.created` events across all authors. Keepalive every 15 seconds.

---

8. Endpoint Summary Table

Method	Path	Auth	Description
`POST`	`/api/v1/auth/login`	None	Login, receive JWT
`GET`	`/api/v1/auth/me`	Any	Current user profile
`GET`	`/api/v1/me/books`	author	Author's books + royalty summary
`GET`	`/api/v1/me/tickets`	author	Author's ticket list
`POST`	`/api/v1/me/tickets`	author	Create new ticket
`GET`	`/api/v1/me/tickets/{id}`	author	Ticket detail (no internal notes)
`POST`	`/api/v1/me/tickets/{id}/messages`	author	Author reply
`GET`	`/api/v1/me/stream`	author	SSE updates for this author
`GET`	`/api/v1/admin/tickets`	admin	Filtered ticket queue
`GET`	`/api/v1/admin/tickets/{id}`	admin	Full ticket detail
`PATCH`	`/api/v1/admin/tickets/{id}`	admin	Update status / category / priority / assignee
`POST`	`/api/v1/admin/tickets/{id}/messages`	admin	Send staff reply
`POST`	`/api/v1/admin/tickets/{id}/notes`	admin	Internal note (hidden from author)
`POST`	`/api/v1/admin/tickets/{id}/draft`	admin	Generate / retrieve AI draft
`POST`	`/api/v1/admin/tickets/{id}/classify`	admin	Retry AI classification
`GET`	`/api/v1/admin/stats`	admin	Dashboard aggregate stats
`GET`	`/api/v1/admin/staff`	admin	List admins (for assignment dropdown)
`GET`	`/api/v1/admin/stream`	admin	SSE updates for all tickets
`GET`	`/health`	None	Liveness probe
`GET`	`/health/ready`	None	DB readiness probe

---

9. Demo Credentials

Role	Email	Password
Author	`priya.sharma@email.com`	`Author2024!`
Author	`kavita.deshmukh@email.com`	`Author2024!`
Author	Authors `AUTH001` – `AUTH010`	`Author2024!`
Admin	`ops.admin@bookleaf.demo`	`Admin2024!`
Admin	`support.lead@bookleaf.demo`	`Admin2024!`

Free-tier Render cold starts may take ~60 seconds on the first request.

---

10. Known Limitations

- No author self-registration — accounts are created via the seed script.
- SSE reconnect logic is client-side; the server does not replay missed events on reconnect.
- `ai_confidence` is stored from the model's own output and should be treated as an indicator only, not a calibrated probability.
- The production URL in this document is a placeholder — update it after deploying to Render.